

## Constraints (envs/main)

**Scope:** The constraint system of the default task Isaac-ConstrainedALBC-TRPO-v0 (constrained\_albc/envs/main/), the attitude-only ALBC policy. Ten IPO constraints ship on this task — **5 probabilistic** (binary violation-probability budgets) + **5 average** (expected-magnitude budgets) — defined in envs/main/config.py and consumed by ConstraintTRPO in algorithms/constraint\_trpo.py.

This is a code-level reference verified against disk. The legacy full-DOF variant (envs/full\_dof/, Isaac-ConstrainedALBC-Full-\*-v0) reuses the same constraint list constant but is a different task and is **not** described here.

---

### 1. Overview

The constraint system is three layers with a clean separation: **definitions** decide what each cost measures, **optimization** decides how a cost budget is enforced, and **wiring** decides which budgets and hyperparameters reach a run.

1. **Definitions (cost functions).** mdp/constraints.py holds 10 per-step cost functions. Each returns a (num\_envs,) non-negative cost tensor. compute\_all\_costs() stacks them into a single (num\_envs, K) tensor every step (constraints.py:326-332).
2. **Optimization (ConstraintTRPO + IPO log-barrier).** constraint\_trpo.py folds an interior-point log-barrier on the K cost budgets directly into the TRPO surrogate objective. There is no separate Lagrangian dual, no projection step — a single joint scalar drives the natural-gradient step.
3. **Wiring (budgets).** config.py binds each cost function to a per-step budget  $D_k$  and a name; the algorithm rescales that to a discounted budget  $d_k = D_k / (1 - \gamma_c)$  (constraint\_trpo.py:146-151) and compares it to the batch's mean discounted cost return.

```
cost function f_k(state) -> compute_all_costs -> costs (num_envs, K) [mdp/constraints.py]
|
v per-step cost, one column per constraint
cost GAE (cost_gamma, cost_lam) -> J_hat_C_k (mean discounted cost return, K-vector)
|
v
IPO adaptive threshold: d_k^i = max(d_k, J_hat_C_k + alpha * d_k) [constraint_trpo.py:306-308]
|
v
barrier margin_k = d_k^i - cost_surr_k # plain difference, NOT / d_k
barrier = -sum_k log(margin_k.clamp(min=1e-8)) / barrier_t [constraint_trpo.py:454-464]
|
v
surrogate L = -E[A * ratio] + barrier - entropy_bonus [constraint_trpo.py:461,480]
|
v
TRPO natural-gradient step under a pure-KL trust region (barrier curvature NOT in Fisher)
```

The crucial structural fact: ConstraintTRPO contains **zero** code that distinguishes probabilistic from average constraints. All K columns flow through byte-identical GAE, standardization, adaptive-threshold, and barrier code. The prob/avg split lives entirely in how each cost signal is computed upstream (mdp/constraints.py) and in documentation convention (config.py), never in how the optimizer consumes it.

---

## 2. Constraint taxonomy — probabilistic vs average

The 10 shipped constraints split 5 + 5. The split is expressed only by list order and inline comments in `config.py` — # --- Probabilistic (5) --- above entries [0:5] and # --- Average (5) --- above entries [5:10] (`config.py:56`, `config.py:62`). There is `no is_probabilistic/kind/type` field on `ConstraintTerm-Cfg`; its only fields are `func`, `params`, `budget`, `name` (`constraints.py:50-57`).

| Category          | Cost signal                                                    | Budget $D_k$ meaning          | Feasibility target                 |
|-------------------|----------------------------------------------------------------|-------------------------------|------------------------------------|
| Probabilistic (5) | binary indicator<br>$\mathbb{1}[\text{violated}] \in \{0, 1\}$ | maximum violation probability | $\mathbb{E}[\mathbb{1}] \leq D_k$  |
| Average (5)       | continuous<br>non-negative cost                                | maximum expected magnitude    | $\mathbb{E}[\text{cost}] \leq D_k$ |

Mathematically, both categories are the same constrained-MDP object  $\mathbb{E}[\sum_t \gamma_c^t c_k(s_t, a_t)] \leq d_k$  — they differ only in what the per-step cost  $c_k$  is. A probabilistic constraint’s  $c_k$  is a  $\{0, 1\}$  indicator, so its discounted sum is (up to the  $\gamma_c$  discount) a violation count, and the budget reads as a probability. An average constraint’s  $c_k$  is a continuous magnitude, so the budget reads as an allowed mean level.

**The mechanism that treats them “differently” is not in the optimizer.** The docstring at `constraints.py:14-24` numbers [0]–[4] as probabilistic and [5]–[9] as average; that ordering plus the underlying cost-function math (indicator vs continuous) is the entire distinction. At runtime `compute_all_costs` (`constraints.py:326-332`) does `torch.stack([t.func(...) for t in cfg.terms], dim=-1)` — identical treatment of all 10 terms. One consequence worth noting for the binary indicators: their per-constraint cost-advantage columns have near-zero variance, so the standardization step floor-clamps std to `min=1.0` specifically to keep those binary channels from blowing up (see Section 4).

## 3. The 10 constraint terms

All values verbatim from `config.py:57-69`. Discounted budget  $d_k = D_k / (1 - \gamma_c)$  with  $\gamma_c = 0.99$ , so  $d_k = 100 \cdot D_k$ .

| # | Name          | Function                               | Key param / threshold                | Budget $D_k$ | Type | Physically limits               |
|---|---------------|----------------------------------------|--------------------------------------|--------------|------|---------------------------------|
| 0 | attitude      | <code>attitude_limit_cost</code>       | <code>limit=1.396 rad (80°)</code>   | 0.01         | prob | roll/pitch magnitude            |
| 1 | arm_torque    | <code>torque_limit_cost</code>         | <code>limit_nm=9.5</code>            | 0.08         | prob | arm joint torque                |
| 2 | arm_joint_vel | <code>velocity_limit_cost</code>       | <code>limit_rad_per_s=4.189</code>   | 0.02         | prob | arm joint speed                 |
| 3 | joint1_pos    | <code>joint1_position_cost</code>      | <code>limit_rad=4*pi (12.566)</code> | 0.01         | prob | joint-1 absolute angle          |
| 4 | cumul_yaw     | <code>cumulative_yaw_limit_cost</code> | <code>limit_rad=8*pi (25.133)</code> | 0.01         | prob | accumulated yaw                 |
| 5 | thruster_util | <code>thruster_utilization_cost</code> | (no params)                          | 0.40         | avg  | rotation thruster authority use |
| 6 | rp_rate       | <code>rp_rate_cost</code>              | <code>soft_threshold=0.5</code>      | 0.10         | avg  | roll/pitch angular rate         |

| # | Name            | Function             | Key param / threshold    | Budget $D_k$ | Type | Physically limits        |
|---|-----------------|----------------------|--------------------------|--------------|------|--------------------------|
| 7 | yaw_rate        | yaw_rate_cost        | soft_threshold=0.55      | 0.10         | avg  | yaw angular rate         |
| 8 | rp_vel_settling | rp_vel_settling_cost | settling_threshold=0.087 | 0.20         | avg  | roll/pitch settling      |
| 9 | manipulability  | manipulability_cost  | w_threshold=0.3          | 0.05         | avg  | arm manipulability floor |

A comment directly above the list records prior surgery: `thruster_rate` was removed (“structurally incompatible with `entropy_coef>0`: noise alone violates 5x”) and `thruster_sat` was reverted to `thruster_util` (average, budget 0.40, “original form”) — `config.py:53–54`.

### 3.1 Probabilistic terms ([0:5])

Each returns a  $\{0, 1\}$  indicator; the discounted-sum budget is a violation probability.

- **attitude** —  $\mathbb{1}[|\text{roll}| > \text{limit} \vee |\text{pitch}| > \text{limit}]$ ,  $\text{limit} = 1.396$  rad.
- **arm\_torque** —  $\mathbb{1}[\max_j |\tau_j| > 9.5 \text{ Nm}]$ .
- **arm\_joint\_vel** —  $\mathbb{1}[\max_j |\dot{q}_j| > 4.189 \text{ rad/s}]$ .
- **joint1\_pos** —  $\mathbb{1}[|\theta_1| > 4\pi]$ , i.e. joint-1 absolute angle past the  $\pm 4\pi$  rail.
- **cumul\_yaw** —  $\mathbb{1}[|\theta_{\text{yaw,cum}}| > 8\pi]$ , accumulated (unwrapped) yaw.

### 3.2 Average terms ([5:10])

Each returns a continuous non-negative cost; the discounted-sum budget is an expected magnitude.

- **thruster\_util** — `thruster_utilization_cost` takes **no configurable threshold** (it relies on `ConstraintTermCfg.params` default `{}`, `constraints.py:55`). Per its docstring (`constraints.py:20`) it returns the per-env maximum absolute normalized thruster state,  $\max_i |s_i|$ . This is the single control-authority channel among the 10.
- **rp\_rate** — soft-thresholded roll/pitch angular-rate cost, hinged at 0.5: cost accrues only above the soft threshold.
- **yaw\_rate** — soft-thresholded yaw-rate cost, hinged at 0.55.
- **rp\_vel\_settling** — the mean roll/pitch angular rate  $(|p| + |q|)/2$ , masked to be nonzero only when the attitude error is within `settling_threshold=0.087` rad (5 deg) of target: cost is zero during transit (att error  $>$  threshold) and active during settling (att error  $\leq$  threshold). The name matches the implementation (`constraints.py:206–228`). Note the caveat that `settling_threshold` gates on the attitude error, not on the angular velocity itself.
- **manipulability** — hinged below a manipulability floor `w_threshold=0.3`; cost accrues when the arm’s manipulability measure drops below the floor.

### 3.3 Experiment-only joint1 terms (not shipped)

Two extra average cost functions exist for the joint1-anti-drift experiment line (Section 7); neither is in the shipped 10 (`constraints.py:26–29`):

- **joint1\_centering\_cost** (arm “A”) —  $\text{wrap}(\theta_1)^2$ , instantaneous wrapped joint-1 angle (`constraints.py:245`).
- **joint1\_cumulative\_cost** (arm “B”) —  $|q_1^{\text{des}} - q_1^{\text{nom}}|$ , the unwrapped running command displacement (`constraints.py:269`, using `env._joint_pos_targets[:, 0] - env._nominal_joint_pos[0]`).

## 4. ConstraintTRPO optimization

Algorithm body: `algorithms/constraint_trpo.py`. ConstraintTRPO is a standalone algorithm (aliased `ALBCCConstraintTRPO`, `rsl_rl_ppo_cfg.py:25`), not an `rsl_rl.PPO` subclass.

### 4.1 IPO log-barrier

The interior-point objective (module docstring form):

$$\begin{aligned} \max_{\pi} \quad & \mathbb{E}[A(s, a)] + \frac{1}{t} \sum_k \log(d_k^i - \hat{J}_{C_k}) \\ \text{s.t.} \quad & \text{KL}(\pi \parallel \pi_i) \leq \delta_{\max} \\ \text{where} \quad & d_k^i = \max(d_k, \hat{J}_{C_k} + \alpha d_k) \end{aligned}$$

In code the barrier is built per update (`constraint_trpo.py:454-464`):

$$\begin{aligned} \text{margin}_k &= \underbrace{(d_k^i - \hat{J}_{C_k})}_{\text{barrier\_base}} - \text{cost\_surr}_k \\ \text{cost\_surr}_k &= \frac{1}{1 - \gamma_c} \mathbb{E}[\text{ratio} \cdot \hat{A}_k^C] \\ \text{barrier} &= -\frac{1}{t} \sum_k \log(\text{margin}_k \cdot \text{clamp}(\min = 10^{-8})) \end{aligned}$$

Two implementation facts matter. First, **margin is a plain difference, never normalized by  $d_k$**  — there is no  $\hat{J}_C/d_k$  division anywhere in the file. Second, the two terms of the margin are on the **same discounted-cost scale but are not multiplied by the same factor in code**: `barrier_base = adaptive_d_k - mean_cost_returns` carries no explicit  $1/(1 - \gamma_c)$ , while `cost_surr` is scaled by `inv_one_minus_gamma` at `constraint_trpo.py:462` (both `mean_cost_returns` and `adaptive_d_k` already live on the discounted-return scale via the budget rescale  $d_k = D_k/(1 - \gamma_c)$ ). The `clamp(min=1e-8)` caps the barrier so it can never literally reach  $-\infty/+\infty$  even at/past infeasibility — this silently softens the interior-point guarantee near the boundary rather than enforcing it hard.

**barrier\_t / barrier\_alpha — the true runtime values (doc-drift warning).** `barrier_t = 100.0` in both the class constructor default (`constraint_trpo.py:64`) and the agent `cfg` (`rsl_rl_ppo_cfg.py:217`) — no drift. **barrier\_alpha has a confirmed drift**: the constructor default is `0.02` (`constraint_trpo.py:65`) but the agent `cfg` — the value that actually reaches every trained run — is `0.05` (`rsl_rl_ppo_cfg.py:218`). The `RslRlConstraintTRPOAlgorithmCfg` field is dispatched to the constructor `kwarg` at build time, so **the effective value is 0.05, and the 0.02 constructor default is dead** whenever the `cfg` supplies the field. Some earlier notes reported `0.02` (or a since-superseded `0.05-vs-0.02` wiki page); the code truth is **0.05, injected from the agent `cfg`**. When in doubt, read the resolved per-run `params/agent.yaml`, not the class signature.

### 4.2 Adaptive threshold and the violations diagnostic

The adaptive threshold is recomputed every update from the current batch’s mean cost returns (`constraint_trpo.py:306-308`, called at `:446`):

$$d_k^i = \max(d_k, \hat{J}_{C_k} + \alpha d_k), \quad \hat{J}_{C_k} = (\text{mean cost return})_k \cdot \text{clamp}(\min = 0)$$

The reported violations monitoring metric compares  $\hat{J}_{C_k}$  against the **raw** budget  $d_k$ , not the adaptive threshold: `violations = (mean_cost_returns - self.d_k) (constraint_trpo.py:447)`. `adaptive_d_k` is used only inside the barrier margin (:454) and the logged barrier margin (:449).

### 4.3 Feasibility, GAE, and standardization

- **Cost GAE.** Per-constraint advantages use the same recursive  $\delta/\lambda$ -return formula as reward GAE but with `cost_gamma=0.99`, `cost_lam=0.95`, a vectorized K-dim accumulator, run reverse in time, followed by a non-finite sanitize that zeros any constraint column containing NaN/Inf across the rollout (`constraint_trpo.py:285-300`). `cost_gamma` must be strictly  $< 1$  or the constructor raises `ValueError (:143-144)`.
- **Time-out bootstrapping.** On time-out (not termination) the per-constraint cost is bootstrapped with `cost_gamma` and the per-constraint value vector, mirroring the reward-side bootstrap but on the cost channel (`constraint_trpo.py:264-266`).
- **Per-constraint cost-advantage standardization (NORBC Sec IV-B).** Each of the K cost-advantage columns is independently mean/std-normalized across the flattened batch dim, with the std floor-clamped to `min=1.0` (`constraint_trpo.py:437-438`). This is a deliberate anti-amplification choice, not the usual  $1e-8$  epsilon: the inline comment (:436) reads `# clamp(min=1.0): binary constraints can have near-zero std, causing 1e8 amplification`. Reward advantages are standardized separately with the ordinary  $1e-8$  guard (:424-426).

### 4.4 TRPO trust-region step

The natural-gradient step operates on the **single combined surrogate scalar** (`constraint_trpo.py:461,480`):

$$L = -\mathbb{E}[A \cdot \text{ratio}] + \text{barrier} - \beta_{\text{ent}} H$$

- **Gradient group.**  $g = \nabla_{\theta} L$  over  $\theta = (\text{actor}, \text{encoder}, \log \sigma)$ .
- **Fisher-vector product uses pure KL curvature only** — a double backprop through the Gaussian KL between old and new policy, plus Tikhonov damping  $Fv + \lambda_{\text{cg}} v$  (`constraint_trpo.py:354-365`). The barrier’s curvature **never** enters the Fisher, so the KL trust region does not “know” about the constraint geometry.
- **Conjugate gradient.** Solves  $Fx = g$  with `cg_iters=10` and an early-exit residual tolerance  $1e-10$  (:367-386).
- **Step size.**  $\Delta\theta = -\sqrt{\delta_{\text{max}}/(\frac{1}{2}\tilde{g}^T g)} \tilde{g}$ ; the update aborts (returns False, no step) if  $\frac{1}{2}\tilde{g}^T g \leq 0$  or non-finite, or if the step direction contains NaN/Inf (:539-547).
- **Line search.** Backtracking, shrink factor 0.5, max 10 backtracks; accepts a step iff the surrogate strictly decreases **and** realized  $\text{KL} \leq \delta_{\text{max}} \cdot \text{line\_search\_kl\_margin}$  with margin 1.5 (:400-410). Note the acceptance bound is **looser** than the  $\delta_{\text{max}}$  used to size the initial step, so an accepted step can exceed the nominal trust-region radius by up to 50%. On exhaustion, params revert to `old_params`.

**True trust-region values.** `max_kl` also drifts: constructor default 0.002 (`constraint_trpo.py:43`) vs agent cfg 0.005 (`rs1_rl_ppo_cfg.py:191`) — the agent cfg (0.005, 2.5× looser) wins. `cg_damping=0.1` and `line_search_kl_margin=1.5` agree between constructor and cfg (no drift).

### 4.5 std clamping

After every `_trpo_step` (unconditional on line-search success), `log_std` is clamped (`constraint_trpo.py:485-491`). At runtime the **per-dim floor branch is the one exercised**, because `min_std_per_dim` is non-empty:

$$\text{min\_std\_per\_dim} = (0.10, 0.10, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05), \quad \text{max\_std} = 2.0$$

Arm dims (indices 0,1) floor at 0.10; thruster dims (2-7) floor at 0.05 (rsl\_rl\_ppo\_cfg.py:236). The scalar min\_std=0.05 agent-cfg field is **dead** at runtime whenever min\_std\_per\_dim is non-empty (which it is by default) — only the per-dim tensor is applied. The constructor’s scalar defaults (min\_std=0.01, min\_std\_per\_dim=()) would take the scalar branch, but the cfg overrides both.

The entropy bonus follows the same per-dim-wins pattern: agent-cfg entropy\_coef\_per\_dim = (0.01, 0.01, 0.001 × 6) (rsl\_rl\_ppo\_cfg.py:229) overrides the scalar entropy\_coef=0.003 (:223), which is therefore also dead at runtime.

#### 4.6 Cost critic

- **Parameter-group split.** Two disjoint groups. The **policy group** (actor + encoder + log\_std) is updated by the TRPO natural gradient — no Adam, no optimizer object; log\_std is deliberately included so the KL trust region bounds noise changes, and the encoder is kept in this group because decoupling it dropped encoder gradients ~85% (code comment, constraint\_trpo.py:154–158). The **value group** (matched by the value\_prefixes tuple critic. / cost\_critic. / value\_backbone. / reward\_head. / cost\_head.) is updated by Adam at value\_lr (:160–186).
- **Single multi-head cost critic.** The cost critic is **one** network producing a K-dimensional vector per observation (evaluate\_costs()), **not** K separate networks. Reward and cost critics are updated jointly in one backward pass: total = value\_loss\_coef ·  $L_V$  + cost\_value\_loss\_coef ·  $L_{V_C}$ , both coefficients 1.0 (constraint\_trpo.py:591–605). The per-constraint MSE is .mean(dim=0) over the batch giving a K-vector, then .mean() across constraints to a scalar (:596–597). Any activation detail of the critic heads lives in the policy class ALBCActorCriticEncoder, not in this file — see the network-architecture reference.

---

### 5. Constraint-margin normalization ( $\hat{J}_C/d_k$ )

The **optimizer stores an absolute margin; any binding/slack judgment must normalize by  $d_k$  first**. This is an analysis rule, not a code behavior: the barrier uses the absolute margin correctly (Section 4.1), but a human or engine reading Constraint/margin/<name> off TensorBoard must divide by  $d_k$  to compare across constraints.

Why it flips sign: the discounted budgets  $d_k = 100 \cdot D_k$  span a **40×** range across the 10 constraints (attitude  $d_k = 1.0$  from  $D_k = 0.01$ , vs thruster\_util  $d_k = 40.0$  from  $D_k = 0.40$ ). A constraint deep in slack but with a large budget has a large absolute margin; a constraint near its budget but with a small budget has a small absolute margin. Reading the raw margin therefore inverts the conclusion.

The correct normalized quantity is

$$\frac{\hat{J}_{C_k}}{d_k} = 1 - \frac{\text{margin}_k}{d_k},$$

valid **only in the slack regime**, where the adaptive floor has not engaged, i.e. where Constraint/viol/<name> = -Constraint/margin/<name> exactly (constraint\_trpo.py:447 vs :449). When  $\hat{J}_{C_k} + \alpha d_k > d_k$  the adaptive threshold engages and  $d_k^i \neq d_k$ , breaking that identity.

**Experimental finding (labeled as such).** A real teacher-run report (report.md:174,180) misread attitude and cumul\_yaw as a “binding family” because their absolute margins looked numerically small — when in fact  $\hat{J}_C/d_k = 0.003$  and 0.000 respectively (the deepest slack), while the genuinely binding thruster\_util ( $\hat{J}_C/d_k \approx 0.87$ ) has a large absolute margin simply because its budget is large. The analysis engine still has this gap: analyze\_training.py:constraint\_margin() (:370–379) returns the absolute margin with no  $d_k$  normalization. Source: omx wiki constraint\_margin\_must\_be\_normalized\_j\_c\_d\_k\_absolute\_margin\_fli.md.

---

## 6. DORAEMON and constraints

DORAEMON is the domain-randomization curriculum. Its interaction with the constraint/feasibility machinery is the point here; the DR mechanics live in marinelab. The main env overrides four fields (config.py:479): `DoraemonCfg(enable=True, kl_ub=0.12, performance_lb=250.0, step_interval=250)`.

- **alpha is a feasibility floor, not a DR lever.** DORAEMON's alpha (cited as 0.5 in the config reasoning comment, config.py:473-474; the literal field default comes from marinelab's `DoraemonCfg`, unverified in the read files) gates whether a curriculum-difficulty step is accepted — it does not directly widen randomization. **Experimental finding:** raising alpha 0.50  $\rightarrow$  0.75 (E5, run `trpo_260606_225859`) was a near-null intervention because `success_rate` stayed saturated at 0.96-0.99 the whole run, so the floor never gated a step. The actually-binding constraint on DR-expansion speed is the per-update trust-region KL (`kl_ub`, pinned at 0.06 in that run). To widen robustness you must move `kl_ub` or DR variance directly, **not** the success floor. Source: `omx wiki doraemon_alpha_is_a_feasibility_floor_not_a_dr_expansion_lever_e.md`.
- **performance\_lb and kl\_ub were raised together by design** (config.py:466-478). `performance_lb` went 68.0  $\rightarrow$  250.0 and `kl_ub` went 0.06  $\rightarrow$  0.12. The calibration used a recon run (`trpo_baseline_260608_160453`, 1146 iter): its DORAEMON episode-return distribution was `min=81.9 / p5=227 / p25=250 / median=264 / p95=291`. With `lb=68` sitting below the minimum return, `success = return >= 68` was always 1, so the feasibility constraint ( $\hat{G} \geq \alpha$ ) was **inert** and the curriculum widened DR unconstrained (no self-pacing feedback). `lb=250` (p25) puts starting `success_rate` at ~0.65 — above `alpha=0.5` so the distribution still expands, but the signal is live again instead of pinned at 1; chosen below the median so a reward plateau cannot drag `success_rate` to 0. `kl_ub` was doubled to widen the distribution fast enough to compensate for the slower expansion the raised `lb` induces. The two levers move together by design: `lb` alone makes DR easier, `kl_ub` alone leaves `success_rate` pinned at 1.
- **success definition.** `success = accumulated_episode_return >= performance_lb`, computed in `albc_env.py` (`_episode_return_accum += reward`; `success = return >= performance_lb`; config.py:467 comment).

`step_interval = 250` sets how often the curriculum updates.

---

## 7. joint1 anti-drift experiment

An off-by-default toggle can append an 11th constraint term to test joint-1 anti-drift. Two fields on `ALBCEnvCfg`:

- `joint1_constraint_arm`: `str = "none"` — one of {"none", "A", "B"} (config.py:532).
- `joint1_constraint_budget`: `float = 0.05` — per-step average budget  $d_k$  for arm A/B (config.py:533).

**Materialization path.** `apply_joint1_constraint_arm()` (constraints.py:301-323) reads the arm; "none" is a no-op; "A" appends a `joint1_centering_cost` term (name `joint1_centering`); "B" appends a `joint1_cumulative_cost` term (name `joint1_cumulative`); anything else raises `ValueError`. It appends via `env_cfg.constraints.terms = [*env_cfg.constraints.terms, term]`.

**Why it is called from `ALBCEnv.__init__`, not a `cfg __post_init__`.** The call site is `albc_env.py:128`, inside `ALBCEnv.__init__` (after payload-viz and DR-sampler init). This is deliberate: `hydra's update_class_from_dict` override lands after the `cfg __post_init__` but before `ALBCEnv.__init__` runs (constraints.py:304-307, `albc_env.py:123-127`). Putting the call in `__post_init__` would always observe `arm="none"` and silently produce a baseline-duplicate run instead of arm A/B.

**Single-mechanism discipline.** For A/B the reward’s `k_joint1_center` must be set to 0.0 so exactly one centering mechanism is active (`config.py:520-524`). This is a documented experimenter requirement — `config.py`’s `ALBCEnvCfg.reward` call does not set it automatically (the `ALBCRewardCfg` default is already 0.0, but nothing enforces it when overriding reward).

**Behavior — never binds under station-keeping (experimental finding).** In a flat-target station-keeping task the joint1-cumulative constraint essentially never binds: the policy naturally parks around **0.36 revolutions**, deeply inside the  $\pm 4\pi$  (2-rev) rail, with the violation metric sitting at **-0.9997** (near-full head-room) every iteration and peak  $|\theta_{\text{cum}}|$  never exceeding  $\sim 1.22$  rev across any of 4 DR difficulty levels (0/64 envs  $> 4\pi$ ). Source: `omx wiki joint1_cumulative_rotation_constraint_never_binds_policy_parks_a.md` (run `trpo_joint1_cumul_rot_260629_183545`). Caveat: the wiki run there frames the term as a probabilistic  $\mathbb{1}[|\theta_{\text{cum}}| > 4\pi] \leq 0.01$  constraint, whereas the shipped-code arm-“B” function `joint1_cumulative_cost` is an average  $|q_1^{\text{des}} - q_1^{\text{nom}}|$  cost — the same experiment line spans two variants across runs; the substance (never binds) holds.

**Generalizes OOD even while not binding (experimental finding).** Despite never binding under station-keeping, the same constraint line keeps cumulative rotational drift bounded out-of-distribution (drift slope  $2.2 \times 10^{-4}$  rad/s, p95 final  $|\text{drift}| = 0.177$  rad  $< 0.224$  rad budget) even while the unrelated attitude tracker develops a genuine OOD heavy tail (roll steady-state error env-median  $0.36^\circ$  vs env-mean  $3.87^\circ$ , worst env  $63^\circ$ ). Source: `omx wiki joint1_cumulative_ipo_constraint_generalizes_drift_bounded_at_oo.md` (run `trpo_cumul_constraint_260627_231709`). Caveat: this page calls it a “binding average-constraint” while the companion page above documents the same line as never binding — the discrepancy is in the source knowledge (likely “binding” used loosely to mean “active/present”), flagged rather than reconciled here.

**Off by default = byte-identical.** With `arm="none"` (the default), `apply_joint1_constraint_arm` returns immediately, the shipped 10-term set is untouched, and `K` stays at 10. Enabling A/B adds exactly one cost-critic head; the MLP layer counts, dims, and activations are unchanged.

## 8. Logging / metrics

Constraint metrics are emitted once per training iteration by `ConstraintEncoderRunner._log_constraint_metrics` (invoked from the overridden `log()`, gated on `self._should_log`; `constraint_encoder_runner.py:248-250`). The algorithm keeps per-step running state (`_last_violations`, `_last_barrier_margins`, `_last_barrier_penalty`) read only for this logging (`constraint_trpo.py:129-133`).

| Metric                     | Meaning                                                                                                                  |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Constraint/viol/<name>     | $\hat{J}_{C_k} - d_k$ against the <b>raw</b> discounted budget ( <code>constraint_trpo.py:447</code> ); negative = slack |
| Constraint/margin/<name>   | $d_k^i - \hat{J}_{C_k}$ (adaptive-threshold margin, :449); <b>absolute</b> , not $d_k$ -normalized                       |
| Constraint/barrier_penalty | aggregate scalar barrier penalty (no per-constraint suffix)                                                              |
| Policy/line_search_success | line-search accept rate this update                                                                                      |
| Policy/entropy             | mean policy entropy                                                                                                      |

The `<name>` suffix is the constraint’s configured name (`ALBCCConstraintCfg.constraint_names`, `constraints.py:77-78`), falling back to the numeric index. The namespace is a 2-level `Constraint/<type>/<name>` hierarchy for `viol/margin` plus the flat `Constraint/barrier_penalty`; DO-RAEMON curriculum metrics use a separate `DORAEMON/*` namespace and policy diagnostics a `Policy/*` namespace.



**Anomaly thresholds** flagged by the analysis engine (analyze\_training.py ANOMALY\_RULES): line\_search\_success < 0.5 FAIL (“TRPO line search failing. Cost gradient may dominate. Check barrier\_t and constraint budgets.”), barrier\_penalty > 0.1 SPIKE; plus general RL-health rules entropy < 0 COLLAPSED, noise\_std < 0.25 LOW / >= 0.95 CEILING, z\_std < 0.1 LOW, grad\_norm < 1e-4 DEAD, roll\_deg > 20 HIGH, pitch\_deg > 25 HIGH. **Correction to a stale wiki note:** the encoder-latent saturation rule uses Encoder/z\_min < -0.98 / Encoder/z\_max > 0.98 in the actual engine code (analyze\_training.py:41-42), **not** the  $\pm 0.95$  some notes state. thruster\_util\_max > 0.95 is cited in the wiki as a saturation anomaly but has no matching ANOMALY\_RULES entry in the engine code — treat it as an analysis heuristic, not a code-encoded rule.

---

## 9. Which constraints actually bind (experimental findings)

Everything in this section is an **experimental result** from prior analysis runs, kept deliberately separate from the code definitions above. Do not treat these as code invariants; they are what specific runs showed.

**Only thruster\_util binds.** Of the 10 shipped constraints, only thruster\_util reaches its discounted budget in practice —  $\hat{J}_C/d_k \approx 0.869$ -0.870 in the teacher run. The other 9 sit in slack, several deeply so: rp\_vel\_settling 0.455, arm\_torque 0.407, rp\_rate 0.319, yaw\_rate 0.138 (slack); manipulability 0.038, arm\_joint\_vel 0.031, joint1\_pos 0.005, attitude 0.003 (deep slack); cumul\_yaw 0.000 (“fully inert”). Source: omx wiki constraint\_margin\_must\_be\_normalized\_j\_c\_d\_k\_absolute\_margin\_fli.md. (Note: the binding figure in the teacher run is  $\sim 0.87$ ; there is no supported 0.94-in-the-teacher value — the 0.944 figure below is the E6 budget-halving run, not the teacher.)

**Budget  $\times 0.5$  starves authority.** The E6 experiment halved all 10 budgets. Only thruster\_util responded, pushing further into binding ( $\hat{J}_C/d_k$  rose  $0.869 \rightarrow 0.944$ ); the other 9 stayed slack. That single additional bind on the control-authority channel caused authority starvation: per-step reward fell 54% (Reward/total 7.96  $\rightarrow$  3.68), lin\_vel reward went negative, and policy entropy collapsed (crossing 0 at iter 2289, an anomaly never seen in the teacher run). **Rule:** tightening a control-authority channel (thruster) is destructive. Source: omx wiki constraint\_budget\_x0\_5\_binds\_only\_thruster\_util\_authority\_starva.md. This binding conclusion is computed in the slack regime ( $\hat{J}_C = d_k - \text{margin}$ ), so it is barrier\_alpha-independent and holds at either 0.02 or 0.05.

**The feasibility constraint was inert when success = return  $\geq 68$ .** With the old performance\_lb=68 sitting below the minimum episode return, success was pinned at 1, the DORAEMON feasibility constraint never fired, and the curriculum widened DR without self-pacing feedback — the exact motivation for the lb 68  $\rightarrow$  250 / kl\_ub 0.06  $\rightarrow$  0.12 recalibration in Section 6.

---

## Source files

- constrained\_albc/envs/main/mdp/constraints.py — 10 cost functions, ConstraintTermCfg, ALBC-ConstraintCfg, compute\_all\_costs, apply\_joint1\_constraint\_arm
- constrained\_albc/envs/main/config.py — ALBCEnvCfg, \_FULL\_DOF\_CONSTRAINT\_TERMS (the shipped 10 budgets), DORAEMON overrides, joint1 toggles
- constrained\_albc/envs/main/config\_noconstraint.py — ALBCNoConstraintEnvCfg (terms=[], TRPO-NoIPO / PPO-Enc ablations)
- constrained\_albc/envs/main/algorithms/constraint\_trpo.py — ConstraintTRPO + IPO barrier, adaptive threshold, cost GAE, TRPO step, std clamp, cost critic
- constrained\_albc/envs/main/agents/rs1\_rl\_ppo\_cfg.py — Rs1RLConstraintTRPOAlgorithmCfg (runtime barrier\_alpha/max\_kl/std/entropy values)

- `constrained_albc/envs/main/runners/constraint_encoder_runner.py` — `_log_constraint_metrics`, `num_constraints` auto-sync
- `.omx/profile/analyze_training.py` — `ANOMALY_RULES`, `_constraint_margin`